arXiv:2411.04965v1 [cs.CL] 7 Nov 2024

BitNet a4.8: 4-bit Activations for 1-bit LLMs

Hongyu Wang* **Shuming Ma*** **Furu Wei**◇

https://aka.ms/GeneralAI

Abstract

Recent research on the 1-bit Large Language Models (LLMs), such as BitNet b1.58 [MWM+24], presents a promising direction for reducing the inference cost of LLMs while maintaining their performance. In this work, we introduce **BitNet a4.8**, enabling 4-bit activations for 1-bit LLMs. BitNet a4.8 employs a hybrid quantization and sparsification strategy to mitigate the quantization errors introduced by the outlier channels. Specifically, we utilize 4-bit activations for inputs to the attention and feed-forward network layers, while sparsifying intermediate states followed with 8-bit quantization. Extensive experiments demonstrate that BitNet a4.8 achieves performance comparable to BitNet b1.58 with equivalent training costs, while being faster in inference with enabling 4-bit (INT4/FP4) kernels. Additionally, BitNet a4.8 activates only 55% of parameters and supports 3-bit KV cache, further enhancing the efficiency of large-scale LLM deployment and inference.

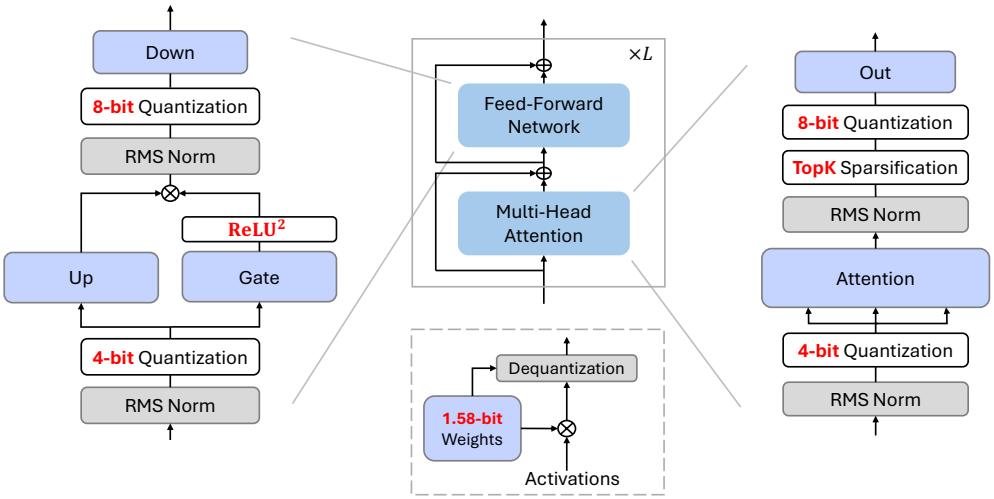

Figure 1: The overview of BitNet a4.8 with both weight and activation quantization. All the parameters are ternery (i.e., 1.58-bit as in BitNet b1.58 [MWM+24]). We use a hybrid quantization and sparsification strategy to deal with outlier activations in certain Transformer sub-layers.

* Equal contribution. ◇ Corresponding author. S. Ma and F. Wei are with Microsoft Research. H. Wang is with University of Chinese Academy of Sciences.

1 Introduction

Recent works [MWM+24] have shown that 1-bit LLMs can match the full-precision models given the same amount of parameters and training tokens while being significantly cost-effective in terms of latency, memory, throughput, and energy consumption. With model weights represented in 1.58-bit (i.e., {-1, 0, 1}), the bottleneck of inference has shifted from the limited memory bandwidth to high computational cost. Low-bit or sparse activations in LLMs served as a promising approach to further reduce the computational budget while maintaining performance on downstream tasks.

One common approach is to utilize activation sparsity [LWD+23, SXZ+24, LPC+24], which reduces the inference FLOPs and the I/O of weight by pruning the activation entries with smaller magnitudes. Sparsification is particularly well-suited for handling activations that exhibit highly imbalanced long-tailed distributions. Recent works [WMWW24] have demonstrated that LLMs with fully sparsely-activated activations can achieve results comparable to dense models while having much less active parameters.

In addition to sparsification, activation quantization is another approach to accelerate the matrix multiplication. However, the optimization of neural networks with low-bit activations is challenging due to the emergence of outlier dimensions as the training progresses and the model size grows. Despite these outliers only account for a very small portion of the activations [DLBZ22, XLS+23], they have much larger magnitude, which leads to significant quantization errors and performance degradation on downstream tasks. Previous works [XLCZ23, AMC+24, LZF+24, LXW+24] mostly utilize Hadamard or learnable rotary transformation to amortize the outlier features into other entries. However, they are mostly designed for the LLMs of higher precision (e.g., 4-bit). For 1-bit LLMs, the extremely low bit-width of the weights makes it challenging to absorb these transformation matrices directly into the weights, while leaving them as online transformations introduces additional computational overhead and limits overall inference performance.

In this work, we introduce **BitNet a4.8**, a hybrid quantization and sparsification strategy that enables 4-bit activations for 1-bit LLMs. By carefully analyzing the activation distribution of 1-bit LLMs, we selectively apply 4-bit quantization or sparsification based on the distribution patterns of these activations. Specifically, as shown in Figure 1, BitNet a4.8 employs 4-bit activations for the inputs to attention and FFN, while utilizing sparsification with 8 bits for intermediate states. To improve the training efficiency, BitNet a4.8 is trained from 8-bit to 4-bit activations with a two-stage recipe, which requires only a few training tokens to adapt BitNet b1.58 to the low-bit activations at the end of training. Extensive experiments demonstrate that BitNet a4.8 achieves competitive performance to BitNet b1.58 with the same training cost while being significantly more efficient at inference time. Furthermore, BitNet a4.8 has only 55% activated parameters and supports 3-bit KV cache, which further enhances the efficiency of LLM deployment.

2 BitNet a4.8

2.1 Architecture

As shown in Figure 1, BitNet a4.8 adopts the same layout as BitNet b1.58. Following [WMD+23, MWM+24], we replace the linear projections in both attention and feed-forward network (FFN) with BitLinear to learn 1.58-bit weights from the scratch. For activations, we adopt a hybrid quantization and sparsification strategy to mitigate the error introduced by outlier dimensions.

Figure 2 illustrates the distribution of each component's inputs of a BitNet b1.58 model with 7B model size. Inputs to the attention and FFN layers typically follow a Gaussian-like distribution, while activations before the FFN down projections and the output projections in attention have more outlier channels and massive amount of entries around zero. [LPC+24] also reported similar observations for full-precision LLMs. As shown in Figure 3, directly applying low-bit quantization to these intermediate states introduces substantial quantization errors.

Therefore, we use the sparsification method from Q-Sparse [WMWW24] to retain these intermediate states at 8 bits while removing the computation bottleneck. For output projection of self-attention layers, we use a sparsify-then-quantize function:

$$\mathbf{Y} = (\mathrm{Q}_{\mathrm{INT8}}(\mathbf{X}) \odot \mathbf{M}) \cdot \mathrm{Q}_w(\mathbf{W})^T, \ \mathbf{M} = \mathrm{Top}_k(|\mathbf{X}|) \tag{1}$$

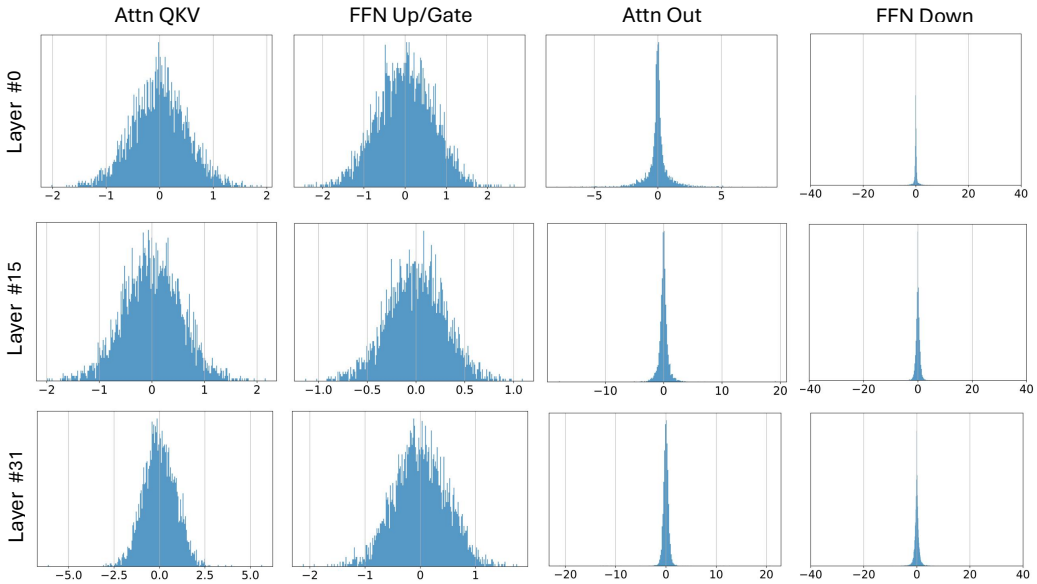

Figure 2: The distribution of the inputs to each projection. The visualization is conducted with a 7B BitNet b1.58 model on a subset of the valid set of C4. For the layers that exhibit Gaussian-like distributions, we employ 4-bit activation quantization. For the layers which distributions are sharp, we adopt Q-Sparse [WMWW24] to perform sparsification on the activations.

where $\mathrm{Q}_w(\cdot)$ and $\mathrm{Q}_{\mathrm{INT8}}(\cdot)$ denote the quantization function for weight $\mathbf{W}$ and activations $\mathbf{X}$, respectively. $\mathbf{M}$ is the mask tensor that indicates the maximum top-K elements in terms of the absolute values of the activations $\mathbf{X}$, $\odot$ is the element-wise multiplication operation.

Specifically, the functions of weight quantization and activation quantization can be formulated as:

$$\mathrm{Q}_w(\mathbf{W}) = \alpha \mathrm{RoundClip}(\frac{\mathbf{W}}{\alpha+\epsilon}, -1, 1),\ \alpha = \mathrm{mean}(|\mathbf{W}|) \tag{2}$$

$$\mathrm{Q}_{\mathrm{INT8}}(\mathbf{X}) = \frac{\gamma}{127}\mathrm{RoundClip}(\frac{127}{\gamma+\epsilon}\mathbf{X}, -128, 127),\ \gamma = \max(|\mathbf{X}|) \tag{3}$$

$$\mathrm{RoundClip}(X, a, b) = \min(\max(\mathrm{round}(X), a), b) \tag{4}$$

For FFN, we adopt squared ReLU [SML+21, WMWW24] and gated linear unit (GLU) to further boost the activation sparsity. It is defined as follows:

$$\mathrm{ReLU}^2\mathrm{GLU}(\mathbf{X}) = \mathbf{X}\mathbf{W}_{\mathrm{up}}^T \odot \mathrm{ReLU}^2(\mathbf{X}\mathbf{W}_{\mathrm{gate}}^T) \tag{5}$$

According to our preliminary experiments, with squared ReLU, the inputs to the down projection achieve over 80% sparsity with minimal impact on performance. Additionally, we observe that the outputs of gate projection $\mathrm{ReLU}^2(\mathbf{X}\mathbf{W}_{\mathrm{gate}}^T)$ exhibit high activation sparsity as well (e.g., 67.5% for 7B models). This characteristic enables further reduction in inference FLOPs for the up projection by first computing the gate projection and then performing the up projection only on the non-zero channels of the gates.

For the input to attention and FFN, since they have much less outlier features, we use absmean function to quantize the activations to 4-bit integers:

$$\mathbf{Y} = \mathrm{Q}_{\mathrm{INT4}}(\mathbf{X}) \cdot \mathrm{Q}_w(\mathbf{W})^T \tag{6}$$

$$\mathrm{Q}_{\mathrm{INT4}}(\mathbf{X}) = \frac{\beta}{\sqrt{7}}\mathrm{RoundClip}(\frac{\sqrt{7}}{\beta+\epsilon}\mathbf{X}, -8, 7),\ \beta = \mathrm{mean}(|\mathbf{X}|) \tag{7}$$

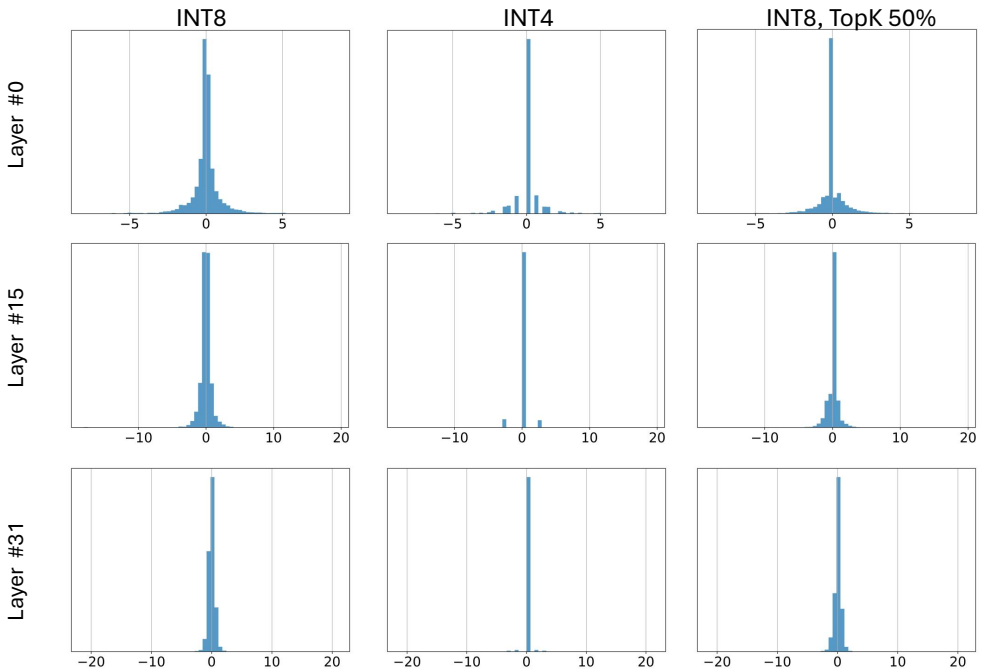

Figure 3: The distribution of the inputs to the output projection of attention with different quantization and sparsification. The visualization is conducted with a 7B BitNet b1.58 model on a subset of the valid set of C4.

2.2 Training

Continue-training from BitNet b1.58. BitNet a4.8 is trained with a two-stage recipe from W1.58A8 to W1.58A4. For the first stage, we train the model with 8-bit activations and ReLU^{2}GLU. For the second stage, we adopt the hybrid quantization and sparsification as shown in Section 2.1. BitNet a4.8 quickly adapts to 4-bit and sparse activations with only a few training tokens while having negligible loss on performance.

Gradient approximation. Following [WMD+23, WMWW24], we use straight-through estimator (STE) [BLC13] to conduct the gradient approximation for BitNet a4.8, as well as mixed precision training to update the parameters. We directly bypass the non-differentiable functions, including the quantization function and top-K sparsification function during the backward propagation. For mixed precision training, we maintain a full-precision latent weight to accumulate parameter updates. During the forward, we quantize the latent weight into 1.58-bit on the fly.

2.3 Floating-point quantization

Floating-point quantization offers a broader dynamic range than the integer-based quantization, which is crucial for handling the long-tailed distribution of the activations. For floating-point precision, we only leave the inputs to down projection of FFN at 8-bit integers, and quantize the other activations to FP4 using MinMax quantizer [LLH+23]. It is defined as follows:

$$\mathrm{Q}_{\mathrm{FP4}}(\mathbf{X}) = \frac{\gamma}{2^{M+b}}\mathrm{Round}(\frac{2^{M+b}}{\gamma}\mathbf{X}),\ \ \gamma = 2^{\max(\lfloor\lfloor\log_2|\mathbf{X}|\rfloor+b\rfloor,1)} \tag{8}$$

$$b = \log_2(\frac{2-2^{-M}}{|\mathbf{X}|_{\max}}) + 2^E - 1 \tag{9}$$

where E and M denote the bit-width of the exponent and mantissa component, respectively. We adopt the E2M1 format due to its larger dynamic range. As shown in Table 1, BitNet a4.8 with FP4

Models	Size	PPL↓	ARCc↑	ARCe↑	HS↑	PQ↑	WGe↑	Avg↑
LLaMA LLM	700M	11.44	27.13	43.27	44.70	68.12	53.99	47.44
BitNet b1.58		12.32	25.00	42.68	42.08	66.97	54.14	46.17
BitNet a4.8 (FP4)		12.40	25.17	42.68	42.36	66.27	52.96	45.89
BitNet a4.8		12.40	25.17	41.58	42.44	66.38	53.04	45.72
LLaMA LLM	1.3B	10.82	27.90	45.16	47.65	69.91	53.35	48.79
BitNet b1.58		11.27	27.65	45.33	46.86	68.39	54.06	48.46
BitNet a4.8 (FP4)		11.38	28.50	44.36	47.03	68.61	54.06	48.51
BitNet a4.8		11.35	28.50	44.15	46.98	68.34	54.14	48.42
LLaMA LLM	3B	9.61	29.95	48.11	55.25	71.76	57.46	52.51
BitNet b1.58		9.97	29.27	49.41	54.42	70.89	57.54	52.30
BitNet a4.8 (FP4)		9.99	29.10	49.24	54.60	71.38	56.12	52.08
BitNet a4.8		9.97	28.33	49.58	54.62	71.16	54.38	51.61
LLaMA LLM	7B	9.20	33.36	51.22	58.33	73.34	58.41	54.93
BitNet b1.58		9.24	32.00	50.88	59.79	72.96	59.83	55.09
BitNet a4.8 (FP4)		9.42	31.57	51.22	58.20	72.47	59.59	54.61
BitNet a4.8		9.37	31.66	50.88	58.78	73.01	59.35	54.74

Table 1: Perplexity and results of BitNet a4.8, BitNet b1.58 and LLaMA LLM on the end tasks. The standard variance of error for average scores is 1.06%.

quantization has the similar performance as it with the hybrid quantization and sparsification strategy based on integers.

3 Experiments

We compared BitNet a4.8 to BitNet b1.58 and our reproduced FP16 LLaMA LLM of various sizes. For 1.58-bit models, we adopted the two-stage weight decay and learning rate scheduling following the training recipe of BitNet b1.58 [MWM+24]. More details can be found in the Appendix A. All models were trained with 100B tokens from the RedPajama dataset [Com23] to ensure a fair comparison. For BitNet a4.8, we first train the model with 8-bit activations for 95B tokens. Then we reuse the optimizer states and continue-train the model with the proposed hybrid quantization and sparsification for 5B tokens. We set topK as 50% for the output projection of attention.

We evaluated the zero-shot accuracy for these models on a range of language tasks using the *lm-evaluation-harness* toolkit [GTA+24], including ARC-Easy (ARCe) [YBS19], ARC-Challenge (ARCc) [YBS19], Hellaswag (HS) [ZHB+19], Winogrande (WGe) [SBBC20] and PIQA (PQ) [BZB+19]. We also reported the perplexity on the validation set of C4 [RSR+19] dataset.

3.1 Main Results

Table 1 summarizes the detailed results of BitNet a4.8, BitNet b1.58 and FP16 LLaMA LLM. The performance gap between full-precision (i.e., FP16) LLaMA LLM and BitNet b1.58 narrows as the model size grows. For 7B models, BitNet b1.58 matches LLaMA LLM in terms of both language model perplexity and average accuracy on the end tasks. Furthermore, BitNet a4.8 achieves performance comparable to BitNet b1.58, with almost no loss in average accuracy.

Sparsity. Table 2 demonstrates detailed sparsity of each component for BitNet a4.8, BitNet b1.58 and FP16 LLaMA LLM across various sizes. The sparsity is calculated with non-embedding parameters on the valid set of C4. Notably, BitNet a4.8 achieves significantly higher sparsity than both BitNet b1.58 and LLaMA LLM. For example, in the 7B model, BitNet a4.8 reaches an overall sparsity of 44.5%, with only 3.4B active parameters. The inputs to the down projection demonstrate particularly high sparsity, consistent with our observation that intermediate state distributions are sharply centered around zero. Additionally, we also observe that the outputs of gate projection are very sparse. It leads to a high sparsity for up projection, since we only need to perform projection on the non-zero channels selected from the gates. Specifically, for the 7B BitNet a4.8, the sparsity of the

Models	Activated	QKV	Out	Up	Gate	Down	Overall
LLaMA LLM	679M	0.0	0.0	0.0	0.0	0.0	0.0
BitNet b1.58	638M	1.2	5.9	1.2	1.2	21.8	6.2
BitNet a4.8	390M	12.1	50.0	66.2	12.1	80.9	42.5
LLaMA LLM	1.2B	0.0	0.0	0.0	0.0	0.0	0.0
BitNet b1.58	1.1B	1.3	5.8	1.2	1.2	22.8	6.4
BitNet a4.8	0.7B	12.0	50.0	65.9	12.1	81.8	42.7
LLaMA LLM	3.2B	0.0	0.0	0.0	0.0	0.0	0.0
BitNet b1.58	3.0B	1.4	7.1	1.3	1.3	30.0	8.2
BitNet a4.8	1.8B	12.1	50.0	70.7	12.1	85.6	44.7
LLaMA LLM	6.5B	0.0	0.0	0.0	0.0	0.0	0.0
BitNet b1.58	6.0B	1.7	11.2	1.4	1.4	24.2	7.3
BitNet a4.8	3.4B	12.1	50.0	71.4	12.0	84.2	44.5

Table 2: Detailed sparsity of BitNet a4.8, BitNet b1.58 and LLaMA LLM on the valid set of C4.

Models	Size	ARCc↑	ARCe↑	HS↑	PQ↑	WGe↑	Avg↑
BitNet a4.8	3B	28.33	49.58	54.62	71.16	54.38	51.61
w/ 4-bit KV		28.24	48.86	54.41	71.87	55.49	51.77
w/ 4-bit QKV		27.30	48.91	54.32	71.98	56.75	51.85
w/ 4-bit Q, 3-bit KV		28.84	48.91	53.87	70.95	56.35	51.78
BitNet a4.8	7B	31.66	50.88	58.78	73.01	59.35	54.74
w/ 4-bit KV		31.40	50.93	58.68	73.12	60.85	55.00
w/ 4-bit QKV		30.63	51.30	58.45	72.52	59.83	54.55
w/ 4-bit Q, 3-bit KV		31.14	50.93	58.07	72.96	59.04	54.43

Table 3: Detailed results of BitNet a4.8 with QKV states varying bit-widths on the end tasks. We reported the zero-shot accuracy of all models.

gates and the inputs to up projection is 67.5% and 12.0%, respectively. Consequently, the sparsity of up projection can be estimated as $1 - (1 - 12.0\%) \times (1 - 67.5\%)$, that is 71.4%.

Low-bit Attention. Table 3 presented detailed results of BitNet a4.8 with low-bit attention in 3B and 7B model size. Low-bit attention is essential for efficient long sequence modeling, as it reduces the memory footprint and IO of KV cache and accelerates the attention computation. In our experiments, we adopted post-RoPE quantization. The QKV heads were directly quantized to unsigned integers using the absmax function, without the need of any calibration dataset. For 3-bit KV quantization, we retain the heads of the bos token at 4-bit, as it contains more outlier features. As shown in Table 3, BitNet a4.8 achieves negligible accuracy loss with 4-bit KV or QKV heads in 3B and 7B models. Furthermore, the KV cache of BitNet a4.8 can be quantized to 3-bit integers, resulting in almost no degradation on average accuracy.

3.2 Ablation Study

Hybrid architecture. Figure 4 presented the training loss curve of 700M BitNet a4.8 with the full INT4/FP4 quantization, and the hybrid quantization and sparsification. We train these models with the first-stage scheduling for 25B tokens from the RedPajama dataset. We adopt absmean and MinMax quantizer for full INT4 and FP4 quantization, respectively. Besides, for full INT4 quantization, we use absmean quantizer with $\beta = 2\text{mean}(|X|)$ for down projection in FFN, as its inputs have larger outliers. As shown in Figure 4, the full INT4 quantization leads to divergence. Furthermore, the hybrid architecture significantly outperforms the full FP4 architecture in terms of training perplexity.

Down projection of FFN. We compared 1.3B BitNet a4.8 with different quantization or activation function for the down projection of FFN. All models were trained with the first-stage scheduling for 50B tokens from the RedPajama dataset. To ensure a fair comparison, we leave the other activations

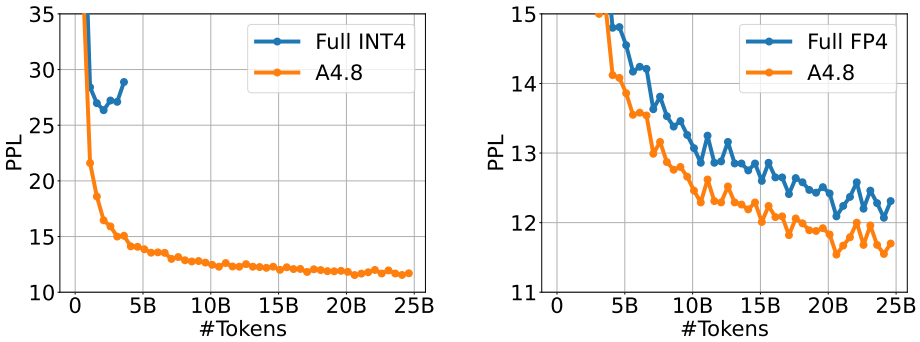

Figure 4: Ablation study on the hybrid quantization and sparsification.

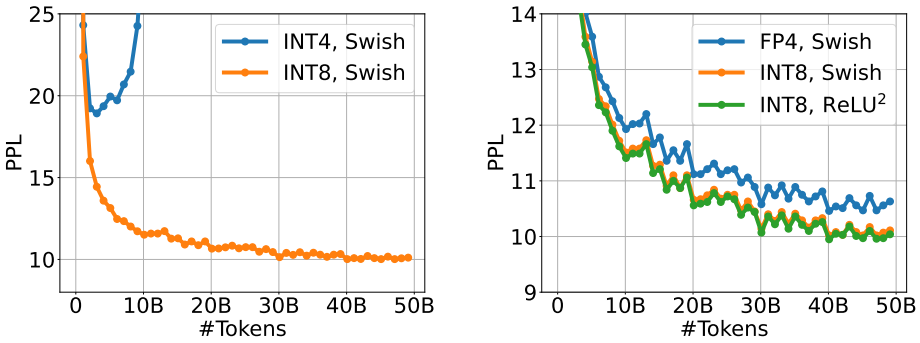

Figure 5: Ablation study on different quantization or activation function for the inputs to down projection of FFN.

at 8-bits. We adopt the absmax quantizer for INT8 quantization and MinMax quantizer for FP4 quantization. The β of absmean quantizer is set as $2\text{mean}(|X|)$. Figure 5 shows the training loss curves of these models. Squared ReLU achieves slightly better training perplexity than Swish while enabling higher sparsity. Furthermore, applying FP4 quantization for the inputs to the down projection leads to a significant performance degradation, while using INT4 activations with STE causes divergence.

Output projection of attention. Table 4 demonstrates detailed results of 3B BitNet a4.8 with and without Top-K sparsification for the inputs to the output projection of attention. Both models were trained with the same two stage recipe from 8-bit to 4-bit activations. We set K as 50% for sparsification. The baseline utilized the INT8 absmax quantizer for the output projection's inputs. The results show that TopK sparsification brings negligible perplexity and accuracy loss.

4-bit quantization. We presented the loss curves of 3B BitNet a4.8 with different 4-bit quantizers for the inputs to the attention and FFN. We compared the performance of BitNet a4.8 with floating-point quantization with E2M1 and E1M2 formats using MinMax quantizer, integer quantization with absmax and absmean quantizer. As shown in Figure 6, FP4 with E2M1 format and INT4 with absmean quantizer achieve slightly better training perplexity, as they are well-suited for handling small-magnitude activation entries.

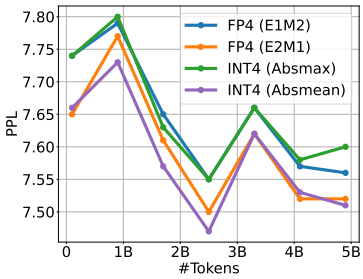

Figure 6: Ablations on 4-bit quantizers for the inputs to attention and FFN.

Quantization	Sparsification	PPL↓	ARCc↑	ARCe↑	HS↑	PQ↑	WGe↑	Avg↑
INT8	-	9.95	28.33	48.53	54.90	72.31	56.51	52.11
INT8	TopK 50%	9.97	28.33	49.58	54.62	71.16	54.38	51.61

Table 4: Ablations on the TopK sparsification for the inputs to the output projection of attention.

Models	HS	PQ	WGe	OBQA	Lambada	MMLU	ARCc	ARCe	Avg
BitNet b1.58 2B	68.66	77.09	62.58	41.40	63.36	50.29	47.61	70.74	60.22
BitNet a4.8 2B	68.21	76.55	64.40	40.60	63.75	50.30	46.59	70.00	60.05

Table 5: Results of BitNet a4.8 and BitNet b1.58 with 2B parameters and 2T training tokens.

3.3 More Training Tokens

Prior research [DLBZ22] demonstrates a positive correlation between the number of training tokens and the prevalence of activation outliers in language models. To rigorously evaluate the scalability characteristics of BitNet a4.8, we conducted extensive experiments using a model configuration with 2 billion parameters trained on 2 trillion tokens. We performed a controlled comparison against BitNet b1.58 using identical training data and configurations. The empirical results, presented in Table 5, demonstrate that BitNet a4.8 maintains performance parity with negligible degradation in accuracy metrics while achieving 4-bit activation compression. These findings provide strong evidence for the efficacy of our proposed approach at scale.

4 Conclusion

In this paper, we present BitNet a4.8 which enables 4-bit activations for 1-bit LLMs. BitNet a4.8 uses a novel hybrid quantization and sparsification architecture to reduce the quantization errors introduce by outlier channels of activations. Specifically, we employ 4-bit quantization for inputs to the attention and FFN layers, while sparsifying the intermediate states with 8-bit integers. BitNet a4.8 is continue-trained from W1.58A8 to W1.58A4. Experimental results demonstrate that BitNet a4.8 achieves results comparable to BitNet b1.58 with the same training cost, while significantly enhancing inference efficiency.

5 Acknowledgements

We would like to acknowledge Lei Wang for the discussion on the inference efficiency.

References

[AMC+24] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *CoRR*, abs/2404.00456, 2024.

[BLC13] Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *CoRR*, abs/1308.3432, 2013.

[BZB+19] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: reasoning about physical commonsense in natural language. *CoRR*, abs/1911.11641, 2019.

[Com23] Together Computer. Redpajama: an open dataset for training large language models, 2023.

[DLBZ22] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *CoRR*, 2022.

[GTA+24] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 07 2024.

[LLH+23] Shih-Yang Liu, Zechun Liu, Xijie Huang, Pingcheng Dong, and Kwang-Ting Cheng. LLM-FP4: 4-bit floating-point quantized transformers. In *EMNLP 2023*, pages 592–605. Association for Computational Linguistics, 2023.

[LPC+24] James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-free activation sparsity in large language models. *CoRR*, abs/2408.14690, 2024.

[LWD+23] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Ré, and Beidi Chen. Deja vu: Contextual sparsity for efficient llms at inference time. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 22137–22176. PMLR, 2023.

[LXW+24] Haokun Lin, Haobo Xu, Yichen Wu, Jingzhi Cui, Yingtao Zhang, Linzhan Mou, Linqi Song, Zhenan Sun, and Ying Wei. Duquant: Distributing outliers via dual transformation makes stronger quantized llms. *arXiv preprint arXiv:2406.01721*, 2024.

[LZF+24] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. Spinquant: LLM quantization with learned rotations. *CoRR*, abs/2405.16406, 2024.

[MWM+24] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *CoRR*, abs/2402.17764, 2024.

[RSR+19] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *CoRR*, abs/1910.10683, 2019.

[SBBC20] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: an adversarial winograd schema challenge at scale. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence*, pages 8732–8740, 2020.

[SML+21] David R. So, Wojciech Manke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V. Le. Primer: Searching for efficient transformers for language modeling. *CoRR*, abs/2109.08668, 2021.

[SXZ+24] Yixin Song, Haotong Xie, Zhengyan Zhang, Bo Wen, Li Ma, Zeyu Mi, and Haibo Chen. Turbo sparse: Achieving llm sota performance with minimal activated parameters. *arXiv preprint arXiv:2406.05955*, 2024.

[WMD+23] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *CoRR*, abs/2310.11453, 2023.

[WMWW24] Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-sparse: All large language models can be fully sparsely-activated. *CoRR*, abs/2407.10969, 2024.

[XLCZ23] Haocheng Xi, Changhao Li, Jianfei Chen, and Jun Zhu. Training transformers with 4-bit integers. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems*, 2023.

[XLS+23] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, 2023.

[YBS19] Vikas Yadav, Steven Bethard, and Mihai Surdeanu. Quick and (not so) dirty: Unsupervised selection of justification sentences for multi-hop question answering. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *EMNLP-IJCNLP*, 2019.

[ZHB+19] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: can a machine really finish your sentence? In *Proceedings of the 57th Conference of the Association for Computational Linguistics*, pages 4791–4800, 2019.

A Hyper-parameters

Size	Hidden Size	GLU Size	#Heads	#Layers	Batch Size	# Tokens	Seq Length
700M	1536	4096	24	24	1M	100B	2048
1.3B	2048	5460	32	24	1M	100B	2048
3B	3200	8640	32	26	1M	100B	2048
7B	4096	11008	32	32	1M	100B	2048

Table 6: Model configurations for both BitNet a4.8, BitNet b1.58 and LLaMA LLM.

Model	Size	Learning Rate	Weight Decay	Warm-up	Adam β
BitNet a4.8	700M	$1.5 \times 10^{-3} \rightarrow 1 \times 10^{-3}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)
	1.3B	$1.2 \times 10^{-3} \rightarrow 8 \times 10^{-4}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)
	3B	$1.2 \times 10^{-3} \rightarrow 6.4 \times 10^{-4}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)
	7B	$1 \times 10^{-3} \rightarrow 6 \times 10^{-4}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)
LLaMA LLM	700M	2.5×10^{-4}	0.1	375	(0.9, 0.95)
	1.3B	2.0×10^{-4}	0.1	375	(0.9, 0.95)
	3B	2.0×10^{-4}	0.1	375	(0.9, 0.95)
	7B	1.5×10^{-4}	0.1	375	(0.9, 0.95)

Table 7: Hyper-parameters for both BitNet a4.8 and LLaMA LLM training.